PYSPARK CODING ASSESSMENT

This assignment focuses on analyzing banking transaction data using the PySpark framework, a powerful big data processing engine built on Apache Spark. The goal is to explore and manipulate large-scale transactional datasets efficiently by leveraging the distributed computing capabilities of PySpark RDDs (Resilient Distributed Datasets).

The analysis involves reading transaction data, transforming it using RDD operations such as `map` and `flatMap`, and performing actions like `collect`, `count`, `first`, `take`, `reduce`, and `saveAsTextFile` to extract insights or persist results. These operations demonstrate PySpark's ability to process data at scale and apply functional programming constructs for high-performance data analytics.

Through this practical implementation, learners gain hands-on experience in:

- Converting structured DataFrames into RDDs,
- Applying transformations to structure and format the data,
- Executing actions to retrieve summaries and intermediate results,
- Saving processed data for further use or reporting.

This assignment provides a foundational understanding of RDD-based programming in PySpark and its application in real-world domains like online banking, where transaction volumes are high and real-time analytics are critical.

```python
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, StringType, DoubleType, DateType
from datetime import date

# Sample data for sales
sales_data = [
    (101, 1, "2023-01-15", 250.75, "Credit Card"),
    (102, 2, "2023-01-17", 540.00, "Cash"),
    (103, 3, "2023-01-18", 300.40, "Debit Card"),
    (104, 1, "2023-02-05", 120.90, "UPI"),
    (105, 2, "2023-02-07", 750.00, "Credit Card"),
    (106, 4, "2023-02-20", 430.50, "Net Banking"),
    (107, 3, "2023-03-01", 210.10, "Cash"),
    (108, 4, "2023-03-05", 610.00, "Debit Card"),
    (109, 1, "2023-03-10", 300.00, "Credit Card")
]

sales_schema = StructType([
    StructField("txn_id", IntegerType(), True),
    StructField("cust_id", IntegerType(), True),
    StructField("txn_date", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("payment_method", StringType(), True)
])

sales_df = spark.createDataFrame(data=sales_data, schema=sales_schema)
```

```python
# Sample data for customers
customers_data = [
    (1, "Alice", "New York", "Gold"),
    (2, "Bob", "California", "Silver"),
    (3, "Charlie", "Texas", "Platinum"),
    (4, "David", "Nevada", "Gold")
]

customers_schema = StructType([
    StructField("cust_id", IntegerType(), True),
    StructField("cust_name", StringType(), True),
    StructField("location", StringType(), True),
    StructField("membership", StringType(), True)
])

customers_df = spark.createDataFrame(data=customers_data, schema=customers_schema)

# Display both DataFrames
print("Sales DataFrame:")
sales_df.show()

print("Customers DataFrame:")
customers_df.show()
```

Sales DataFrame:

txn_id	cust_id	txn_date	amount	payment_method
101	1	2023-01-15	250.75	Credit Card
102	2	2023-01-17	540.0	Cash
103	3	2023-01-18	300.4	Debit Card
104	1	2023-02-05	120.9	UPI
105	2	2023-02-07	750.0	Credit Card
106	4	2023-02-20	430.5	Net Banking
107	3	2023-03-01	210.1	Cash
108	4	2023-03-05	610.0	Debit Card
109	1	2023-03-10	300.0	Credit Card

Customers DataFrame:

cust_id	cust_name	location	membership
1	Alice	New York	Gold
2	Bob	California	Silver
3	Charlie	Texas	Platinum
4	David	Nevada	Gold

PySpark DataFrame Transformations

Category 1: Basic Transformations

1. Select Specific Columns

```
from pyspark.sql.functions import col, lit

# 1. Select specific columns
sales_df.select("txn_id", "amount").show()
```

```
+------+------+
|txn_id|amount|
+------+------+
|   101|250.75|
|   102| 540.0|
|   103| 300.4|
|   104| 120.9|
|   105| 750.0|
|   106| 430.5|
|   107| 210.1|
|   108| 610.0|
|   109| 300.0|
+------+------+
```

2. Add a New Column with withColumn

```
# 2. Add a new column with withColumn
sales_df.withColumn("amount_with_tax", col("amount") * 1.18).show()
```

```
+------+-------+----------+------+--------------+------------------+
|txn_id|cust_id|  txn_date|amount|payment_method|   amount_with_tax|
+------+-------+----------+------+--------------+------------------+
|   101|      1|2023-01-15|250.75|   Credit Card|           295.885|
|   102|      2|2023-01-17| 540.0|          Cash| 637.1999999999999|
|   103|      3|2023-01-18| 300.4|    Debit Card|           354.472|
|   104|      1|2023-02-05| 120.9|           UPI|           142.662|
|   105|      2|2023-02-07| 750.0|   Credit Card|             885.0|
|   106|      4|2023-02-20| 430.5|   Net Banking|507.98999999999995|
|   107|      3|2023-03-01| 210.1|          Cash|247.91799999999998|
|   108|      4|2023-03-05| 610.0|    Debit Card|             719.8|
|   109|      1|2023-03-10| 300.0|   Credit Card|             354.0|
+------+-------+----------+------+--------------+------------------+
```

3. Drop a Column

```
# 3. Drop a column
sales_df.drop("payment_method").show()
```

```
+------+-------+----------+------+
|txn_id|cust_id|  txn_date|amount|
+------+-------+----------+------+
|   101|      1|2023-01-15|250.75|
|   102|      2|2023-01-17| 540.0|
|   103|      3|2023-01-18| 300.4|
|   104|      1|2023-02-05| 120.9|
|   105|      2|2023-02-07| 750.0|
|   106|      4|2023-02-20| 430.5|
|   107|      3|2023-03-01| 210.1|
|   108|      4|2023-03-05| 610.0|
|   109|      1|2023-03-10| 300.0|
+------+-------+----------+------+
```

4. Filter Rows Based on Condition

```
# 4. Filter rows (also shown earlier)
sales_df.filter(col("amount") > 500).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   102|      2|2023-01-17| 540.0|          Cash|
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   108|      4|2023-03-05| 610.0|    Debit Card|
+------+-------+----------+------+--------------+
```

5. Distinct Values in payment_method

```
# 5. Distinct values in payment_method
sales_df.select("payment_method").distinct().show()
```

```
+--------------+
|payment_method|
+--------------+
|   Credit Card|
|          Cash|
|    Debit Card|
|           UPI|
|   Net Banking|
+--------------+
```

6. Rename a Column Using alias

```
# 6. Rename a column using alias
sales_df.select(col("txn_id").alias("transaction_id"), "amount").show()
```

```
+--------------+------+
|transaction_id|amount|
+--------------+------+
|           101|250.75|
|           102| 540.0|
|           103| 300.4|
|           104| 120.9|
|           105| 750.0|
|           106| 430.5|
|           107| 210.1|
|           108| 610.0|
|           109| 300.0|
+--------------+------+
```

7. Drop Duplicates Based on cust_id and payment_method

```
# 7. Drop duplicates based on customer and payment method
sales_df.dropDuplicates(["cust_id", "payment_method"]).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   101|      1|2023-01-15|250.75|   Credit Card|
|   104|      1|2023-02-05| 120.9|           UPI|
|   102|      2|2023-01-17| 540.0|          Cash|
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   107|      3|2023-03-01| 210.1|          Cash|
|   103|      3|2023-01-18| 300.4|    Debit Card|
|   108|      4|2023-03-05| 610.0|    Debit Card|
|   106|      4|2023-02-20| 430.5|   Net Banking|
+------+-------+----------+------+--------------+
```

Category 2: Joins

1. Inner Join

```
# Returns only rows where cust_id exists in both sales_df and customers_df
joined_df = sales_df.join(customers_df, on="cust_id", how="inner")
joined_df.show()
```

```
+-------+------+----------+------+--------------+---------+----------+----------+
|cust_id|txn_id|  txn_date|amount|payment_method|cust_name|  location|membership|
+-------+------+----------+------+--------------+---------+----------+----------+
|      1|   101|2023-01-15|250.75|   Credit Card|    Alice|  New York|      Gold|
|      1|   104|2023-02-05| 120.9|           UPI|    Alice|  New York|      Gold|
|      1|   109|2023-03-10| 300.0|   Credit Card|    Alice|  New York|      Gold|
|      2|   102|2023-01-17| 540.0|          Cash|      Bob|California|    Silver|
|      2|   105|2023-02-07| 750.0|   Credit Card|      Bob|California|    Silver|
|      3|   103|2023-01-18| 300.4|    Debit Card|  Charlie|     Texas|  Platinum|
|      3|   107|2023-03-01| 210.1|          Cash|  Charlie|     Texas|  Platinum|
|      4|   106|2023-02-20| 430.5|   Net Banking|    David|    Nevada|      Gold|
|      4|   108|2023-03-05| 610.0|    Debit Card|    David|    Nevada|      Gold|
+-------+------+----------+------+--------------+---------+----------+----------+
```

2. Left Join

```
# Returns all rows from sales_df and matching rows from customers_df (NULL if no match)
sales_df.join(customers_df, on="cust_id", how="left").show()
```

```
+-------+------+----------+------+--------------+---------+----------+----------+
|cust_id|txn_id|  txn_date|amount|payment_method|cust_name|  location|membership|
+-------+------+----------+------+--------------+---------+----------+----------+
|      1|   101|2023-01-15|250.75|   Credit Card|    Alice|  New York|      Gold|
|      1|   104|2023-02-05| 120.9|           UPI|    Alice|  New York|      Gold|
|      3|   103|2023-01-18| 300.4|    Debit Card|  Charlie|     Texas|  Platinum|
|      2|   102|2023-01-17| 540.0|          Cash|      Bob|California|    Silver|
|      1|   109|2023-03-10| 300.0|   Credit Card|    Alice|  New York|      Gold|
|      3|   107|2023-03-01| 210.1|          Cash|  Charlie|     Texas|  Platinum|
|      4|   106|2023-02-20| 430.5|   Net Banking|    David|    Nevada|      Gold|
|      4|   108|2023-03-05| 610.0|    Debit Card|    David|    Nevada|      Gold|
|      2|   105|2023-02-07| 750.0|   Credit Card|      Bob|California|    Silver|
+-------+------+----------+------+--------------+---------+----------+----------+
```

3. Right Join

```
# Returns all rows from customers_df and matching rows from sales_df (NULL if no match)
sales_df.join(customers_df, on="cust_id", how="right").show()
```

```
+-------+------+----------+------+--------------+---------+----------+----------+
|cust_id|txn_id|  txn_date|amount|payment_method|cust_name|  location|membership|
+-------+------+----------+------+--------------+---------+----------+----------+
|      1|   109|2023-03-10| 300.0|   Credit Card|    Alice|  New York|      Gold|
|      1|   104|2023-02-05| 120.9|           UPI|    Alice|  New York|      Gold|
|      1|   101|2023-01-15|250.75|   Credit Card|    Alice|  New York|      Gold|
|      2|   105|2023-02-07| 750.0|   Credit Card|      Bob|California|    Silver|
|      2|   102|2023-01-17| 540.0|          Cash|      Bob|California|    Silver|
|      3|   107|2023-03-01| 210.1|          Cash|  Charlie|     Texas|  Platinum|
|      3|   103|2023-01-18| 300.4|    Debit Card|  Charlie|     Texas|  Platinum|
|      4|   108|2023-03-05| 610.0|    Debit Card|    David|    Nevada|      Gold|
|      4|   106|2023-02-20| 430.5|   Net Banking|    David|    Nevada|      Gold|
+-------+------+----------+------+--------------+---------+----------+----------+
```

4. Full Outer Join

```
# Returns all rows from both DataFrames; unmatched rows get NULLs in non-matching columns
sales_df.join(customers_df, on="cust_id", how="outer").show()
```

```
+-------+------+----------+------+--------------+---------+----------+----------+
|cust_id|txn_id|  txn_date|amount|payment_method|cust_name|  location|membership|
+-------+------+----------+------+--------------+---------+----------+----------+
|      1|   101|2023-01-15|250.75|   Credit Card|    Alice|  New York|      Gold|
|      1|   104|2023-02-05| 120.9|           UPI|    Alice|  New York|      Gold|
|      1|   109|2023-03-10| 300.0|   Credit Card|    Alice|  New York|      Gold|
|      2|   102|2023-01-17| 540.0|          Cash|      Bob|California|    Silver|
|      2|   105|2023-02-07| 750.0|   Credit Card|      Bob|California|    Silver|
|      3|   103|2023-01-18| 300.4|    Debit Card|  Charlie|     Texas|  Platinum|
|      3|   107|2023-03-01| 210.1|          Cash|  Charlie|     Texas|  Platinum|
|      4|   106|2023-02-20| 430.5|   Net Banking|    David|    Nevada|      Gold|
|      4|   108|2023-03-05| 610.0|    Debit Card|    David|    Nevada|      Gold|
+-------+------+----------+------+--------------+---------+----------+----------+
```

Category 3: Aggregation & Grouping

1. Total Sales Amount

```
from pyspark.sql.functions import sum, avg, max, min, count

# Calculate the total of all transaction amounts
sales_df.agg(sum("amount").alias("total_sales")).show()
```

```
+-----------+
|total_sales|
+-----------+
|    3512.65|
+-----------+
```

2. Average Transaction Amount

```
# Calculate the average transaction amount
sales_df.agg(avg("amount").alias("average_sales")).show()
```

```
+-----------------+
|    average_sales|
+-----------------+
|390.2944444444445|
+-----------------+
```

3. Group by Payment Method and Aggregate Amounts

```
# Group by payment method and compute total, average, max, and min amount
sales_df.groupBy("payment_method").agg(
    sum("amount").alias("total"),
    avg("amount").alias("avg"),
    max("amount").alias("max"),
    min("amount").alias("min")
).show()
```

```
+--------------+-------+-----------------+-----+------+
|payment_method|  total|              avg|  max|   min|
+--------------+-------+-----------------+-----+------+
|   Credit Card|1300.75|433.5833333333333|750.0|250.75|
|          Cash|  750.1|           375.05|540.0| 210.1|
|    Debit Card|  910.4|            455.2|610.0| 300.4|
|           UPI|  120.9|            120.9|120.9| 120.9|
|   Net Banking|  430.5|            430.5|430.5| 430.5|
+--------------+-------+-----------------+-----+------+
```

4. Count Transactions Per Customer

```
# Count number of transactions (rows) for each customer
sales_df.groupBy("cust_id").agg(count("*").alias("txn_count")).show()
```

```
+-------+---------+
|cust_id|txn_count|
+-------+---------+
|      1|        3|
|      3|        2|
|      2|        2|
|      4|        2|
+-------+---------+
```

Category 4: Sorting and Limiting

1. Sort by Amount (Descending Order)

```
# Sort the transactions in descending order of amount
sales_df.orderBy(col("amount").desc()).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   108|      4|2023-03-05| 610.0|    Debit Card|
|   102|      2|2023-01-17| 540.0|          Cash|
|   106|      4|2023-02-20| 430.5|   Net Banking|
|   103|      3|2023-01-18| 300.4|    Debit Card|
|   109|      1|2023-03-10| 300.0|   Credit Card|
|   101|      1|2023-01-15|250.75|   Credit Card|
|   107|      3|2023-03-01| 210.1|          Cash|
|   104|      1|2023-02-05| 120.9|           UPI|
+------+-------+----------+------+--------------+
```

2. Top 3 Highest Transactions

```
# Display the top 3 transactions with the highest amount
sales_df.orderBy(col("amount").desc()).limit(3).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   108|      4|2023-03-05| 610.0|    Debit Card|
|   102|      2|2023-01-17| 540.0|          Cash|
+------+-------+----------+------+--------------+
```

3. Random 30% Sample (Without Replacement)

```
# Take a random 30% sample of the dataset (without replacement)
sales_df.sample(withReplacement=False, fraction=0.3, seed=42).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   104|      1|2023-02-05| 120.9|           UPI|
|   107|      3|2023-03-01| 210.1|          Cash|
|   108|      4|2023-03-05| 610.0|    Debit Card|
+------+-------+----------+------+--------------+
```

Category 5: Set Operations

1. Create a Filtered DataFrame for Demonstration

```
# Make a new DataFrame with transactions where amount > 300
sales_df2 = sales_df.filter(col("amount") > 300).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   102|      2|2023-01-17| 540.0|          Cash|
|   103|      3|2023-01-18| 300.4|    Debit Card|
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   106|      4|2023-02-20| 430.5|   Net Banking|
|   108|      4|2023-03-05| 610.0|    Debit Card|
+------+-------+----------+------+--------------+
```

2. Union of Two DataFrames

```
# Combine sales_df and sales_df2; duplicates will be included
sales_df.union(sales_df2).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   101|      1|2023-01-15|250.75|   Credit Card|
|   102|      2|2023-01-17| 540.0|          Cash|
|   103|      3|2023-01-18| 300.4|    Debit Card|
|   104|      1|2023-02-05| 120.9|           UPI|
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   106|      4|2023-02-20| 430.5|   Net Banking|
|   107|      3|2023-03-01| 210.1|          Cash|
|   108|      4|2023-03-05| 610.0|    Debit Card|
|   109|      1|2023-03-10| 300.0|   Credit Card|
|   102|      2|2023-01-17| 540.0|          Cash|
|   103|      3|2023-01-18| 300.4|    Debit Card|
|   105|      2|2023-02-07| 750.0|   Credit Card|
|   106|      4|2023-02-20| 430.5|   Net Banking|
|   108|      4|2023-03-05| 610.0|    Debit Card|
+------+-------+----------+------+--------------+
```

3. Intersection of Two DataFrames

```
# Return only rows that are present in both DataFrames
sales_df.intersect(sales_df2).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   106|      4|2023-02-20| 430.5|   Net Banking|
|   103|      3|2023-01-18| 300.4|    Debit Card|
|   108|      4|2023-03-05| 610.0|    Debit Card|
|   102|      2|2023-01-17| 540.0|          Cash|
|   105|      2|2023-02-07| 750.0|   Credit Card|
+------+-------+----------+------+--------------+
```

4. Subtract One DataFrame from Another

```
# Return rows that are in sales_df but NOT in sales_df2
sales_df.subtract(sales_df2).show()
```

```
+------+-------+----------+------+--------------+
|txn_id|cust_id|  txn_date|amount|payment_method|
+------+-------+----------+------+--------------+
|   104|      1|2023-02-05| 120.9|           UPI|
|   101|      1|2023-01-15|250.75|   Credit Card|
|   107|      3|2023-03-01| 210.1|          Cash|
|   109|      1|2023-03-10| 300.0|   Credit Card|
+------+-------+----------+------+--------------+
```

Category 6: Window Functions

1. Define Window Specification

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, rank, dense_rank, lag, lead

# Define a window partitioned by customer and ordered by transaction date
window_spec = Window.partitionBy("cust_id").orderBy("txn_date")
```

2. Add row_number, rank, and dense_rank Columns

```
# Add row number, rank, and dense rank based on the window specification
sales_df.withColumn("row_num", row_number().over(window_spec)) \
        .withColumn("rank", rank().over(window_spec)) \
        .withColumn("dense_rank", dense_rank().over(window_spec)) \
        .show()
```

```
+------+-------+----------+------+--------------+-------+----+----------+
|txn_id|cust_id|  txn_date|amount|payment_method|row_num|rank|dense_rank|
+------+-------+----------+------+--------------+-------+----+----------+
|   101|      1|2023-01-15|250.75|   Credit Card|      1|   1|         1|
|   104|      1|2023-02-05| 120.9|           UPI|      2|   2|         2|
|   109|      1|2023-03-10| 300.0|   Credit Card|      3|   3|         3|
|   102|      2|2023-01-17| 540.0|          Cash|      1|   1|         1|
|   105|      2|2023-02-07| 750.0|   Credit Card|      2|   2|         2|
|   103|      3|2023-01-18| 300.4|    Debit Card|      1|   1|         1|
|   107|      3|2023-03-01| 210.1|          Cash|      2|   2|         2|
|   106|      4|2023-02-20| 430.5|   Net Banking|      1|   1|         1|
|   108|      4|2023-03-05| 610.0|    Debit Card|      2|   2|         2|
+------+-------+----------+------+--------------+-------+----+----------+
```

3. Add lag and lead Columns

```
# Add previous and next transaction amount using lag and lead functions
sales_df.withColumn("prev_amount", lag("amount", 1).over(window_spec)) \
        .withColumn("next_amount", lead("amount", 1).over(window_spec)) \
        .show()
```

```
+------+-------+----------+------+--------------+-----------+-----------+
|txn_id|cust_id|  txn_date|amount|payment_method|prev_amount|next_amount|
+------+-------+----------+------+--------------+-----------+-----------+
|   101|      1|2023-01-15|250.75|   Credit Card|       NULL|      120.9|
|   104|      1|2023-02-05| 120.9|           UPI|     250.75|      300.0|
|   109|      1|2023-03-10| 300.0|   Credit Card|      120.9|       NULL|
|   102|      2|2023-01-17| 540.0|          Cash|       NULL|      750.0|
|   105|      2|2023-02-07| 750.0|   Credit Card|      540.0|       NULL|
|   103|      3|2023-01-18| 300.4|    Debit Card|       NULL|      210.1|
|   107|      3|2023-03-01| 210.1|          Cash|      300.4|       NULL|
|   106|      4|2023-02-20| 430.5|   Net Banking|       NULL|      610.0|
|   108|      4|2023-03-05| 610.0|    Debit Card|      430.5|       NULL|
+------+-------+----------+------+--------------+-----------+-----------+
```

4. `map()` : Add ₹100 to each transaction amount

```python
from pyspark.sql import Row

# Create RDD with updated amounts
mapped_rdd = sales_df.rdd.map(lambda row: Row(txn_id=row.txn_id, updated_amount=row.amount + 100))

# Convert to DataFrame
mapped_df = mapped_rdd.toDF()

# Show the updated DataFrame
mapped_df.show()
```

```
+------+--------------+
|txn_id|updated_amount|
+------+--------------+
|   101|        350.75|
|   102|         640.0|
|   103|         400.4|
|   104|         220.9|
|   105|         850.0|
|   106|         530.5|
|   107|         310.1|
|   108|         710.0|
|   109|         400.0|
+------+--------------+
```

5. `flatMap()` : Split payment methods into individual words

```
# Use flatMap to extract words from payment_method
flatmap_rdd = sales_df.rdd.flatMap(lambda row: row.payment_method.split(" "))

# Convert to DataFrame with one column
flatmap_df = flatmap_rdd.map(lambda word: Row(word=word)).toDF()

# Show the DataFrame
flatmap_df.show()
```

```
+-------+
|   word|
+-------+
| Credit|
|   Card|
|   Cash|
|  Debit|
|   Card|
|    UPI|
| Credit|
|   Card|
|    Net|
|Banking|
|   Cash|
|  Debit|
|   Card|
| Credit|
|   Card|
+-------+
```

PySpark RDD Actions

1. `collect()` Action

```
rdd = sales_df.rdd
all_txns = rdd.collect()

for txn in all_txns:
    print(txn)
```

```
Row(txn_id=101, cust_id=1, txn_date='2023-01-15', amount=250.75, payment_method='Credit Card')
Row(txn_id=102, cust_id=2, txn_date='2023-01-17', amount=540.0, payment_method='Cash')
Row(txn_id=103, cust_id=3, txn_date='2023-01-18', amount=300.4, payment_method='Debit Card')
Row(txn_id=104, cust_id=1, txn_date='2023-02-05', amount=120.9, payment_method='UPI')
Row(txn_id=105, cust_id=2, txn_date='2023-02-07', amount=750.0, payment_method='Credit Card')
Row(txn_id=106, cust_id=4, txn_date='2023-02-20', amount=430.5, payment_method='Net Banking')
Row(txn_id=107, cust_id=3, txn_date='2023-03-01', amount=210.1, payment_method='Cash')
Row(txn_id=108, cust_id=4, txn_date='2023-03-05', amount=610.0, payment_method='Debit Card')
Row(txn_id=109, cust_id=1, txn_date='2023-03-10', amount=300.0, payment_method='Credit Card')
```

2. **`.count()` Action**

```
txn_count = sales_df.rdd.count()
print("Total transactions:", txn_count)
```

```
Total transactions: 9
```

3. **`.first()` Action**

```
first_txn = sales_df.rdd.first()
print("First transaction:", first_txn)
```

```
First transaction: Row(txn_id=101, cust_id=1, txn_date='2023-01-15', amount=250.75, payment_method='Credit Card')
```

4.. take**(n) Action**

```
first_5_txns = sales_df.rdd.take(5)
print("First 5 transactions:")
for txn in first_5_txns:
    print(txn)
```

```
First 5 transactions:
Row(txn_id=101, cust_id=1, txn_date='2023-01-15', amount=250.75, payment_method='Credit Card')
Row(txn_id=102, cust_id=2, txn_date='2023-01-17', amount=540.0, payment_method='Cash')
Row(txn_id=103, cust_id=3, txn_date='2023-01-18', amount=300.4, payment_method='Debit Card')
Row(txn_id=104, cust_id=1, txn_date='2023-02-05', amount=120.9, payment_method='UPI')
Row(txn_id=105, cust_id=2, txn_date='2023-02-07', amount=750.0, payment_method='Credit Card')
```

5. **`.reduce()` Action**

```
total_amount = sales_df.rdd.map(lambda row: row.amount).reduce(lambda x, y: x + y)
print("Total amount transacted: ₹", total_amount)
```

```
Total amount transacted: ₹ 3512.65
```

6. **`.saveAsTextFile()` Action**

```
sales_df.rdd.saveAsTextFile("output/customers_text")
```